🏥 Hospital Appointment System – Complete Spring Boot Project Guide

📌 Project Topic

P04: Hospital Appointment System

Entities

- Patient
- Doctor
- Appointment

Relationships

- One Patient can have many Appointments (`OneToMany`)
- One Doctor can have many Appointments (`OneToMany`)
- Each Appointment belongs to one Patient and one Doctor (`ManyToOne`)

Core Requirements

- CRUD for Patient
- CRUD for Doctor
- Book Appointment
- Prevent Overlapping Appointments
- Filter Appointments by Date and Doctor (Bonus)

🤖 Master Prompt for ChatGPT / GitHub Copilot

Copy and paste the prompt below into ChatGPT or GitHub Copilot to generate your full project.

```
Act as a senior Spring Boot instructor and build a complete backend project for
a Hospital Appointment System.

## Technology Stack
- Java 17+
- Spring Boot 3.x
- Spring Web
- Spring Data JPA
- MySQL
- Lombok
- Validation
```

```
- DTO Pattern
- Global Exception Handling

## Architecture
- controller
- service
- repository
- dto
- entity
- exception
- mapper (optional)
- config (optional)

## Entities

### Patient
- id (Long)
- firstName (String, not blank)
- lastName (String, not blank)
- email (String, unique, valid email)
- phoneNumber (String, not blank)
- dateOfBirth (LocalDate)
- gender (String)

### Doctor
- id (Long)
- firstName (String, not blank)
- lastName (String, not blank)
- specialization (String, not blank)
- email (String, unique, valid email)
- phoneNumber (String)

### Appointment
- id (Long)
- appointmentDate (LocalDate)
- startTime (LocalTime)
- endTime (LocalTime)
- status (Enum: SCHEDULED, COMPLETED, CANCELLED)
- notes (String)
- patient (ManyToOne)
- doctor (ManyToOne)

## Relationships
- Patient OneToMany Appointment
- Doctor OneToMany Appointment
- Appointment ManyToOne Patient
- Appointment ManyToOne Doctor
```

```
## DTOs
Create Request and Response DTOs for all entities.

## CRUD APIs

### Patient APIs
POST /api/patients
GET /api/patients
GET /api/patients/{id}
PUT /api/patients/{id}
DELETE /api/patients/{id}

### Doctor APIs
POST /api/doctors
GET /api/doctors
GET /api/doctors/{id}
PUT /api/doctors/{id}
DELETE /api/doctors/{id}

### Appointment APIs
POST /api/appointments
GET /api/appointments
GET /api/appointments/{id}
PUT /api/appointments/{id}
DELETE /api/appointments/{id}
GET /api/appointments/doctor/{doctorId}
GET /api/appointments/date/{date}
GET /api/appointments/doctor/{doctorId}/date/{date}

## Business Logic
When creating or updating an appointment:
1. Check whether patient exists.
2. Check whether doctor exists.
3. Ensure startTime is before endTime.
4. Prevent overlapping appointments for the same doctor.
5. Save appointment if valid.

### Overlap Rule
Two appointments overlap if:
newStart < existingEnd AND newEnd > existingStart

## Validation
Use @Valid and annotations such as:
- @NotBlank
- @Email
- @NotNull
- @FutureOrPresent
```

```
## Exception Handling
Create:
- ResourceNotFoundException
- DuplicateResourceException
- AppointmentConflictException
- GlobalExceptionHandler

## JSON Response Format
{
  "timestamp": "2026-05-19T10:00:00",
  "status": 200,
  "message": "Success",
  "data": {}
}

## Repository Methods
- findByDoctorId(Long doctorId)
- findByAppointmentDate(LocalDate date)
- findByDoctorIdAndAppointmentDate(Long doctorId, LocalDate date)

## application.properties
- MySQL configuration
- Hibernate ddl-auto=update
- show-sql=true

## Additional Requirements
- Use Lombok annotations
- Clean package structure
- Proper comments
- Ready-to-run project
- Include sample Postman requests and JSON examples

Generate all files step by step:
1. pom.xml
2. application.properties
3. entities
4. DTOs
5. repositories
6. services and implementations
7. controllers
8. exceptions
9. GlobalExceptionHandler
10. Sample JSON requests
```

👥 Suggested Team Member Division (9 Members)

> ✅ Recommended workflow: One member creates the base Spring Boot project and pushes it to GitHub. Every member creates their own branch, completes their assigned task, commits regularly, and opens a Pull Request to merge into `main`.

Member	Responsibility	What to Implement	Suggested Branch
Member 1	Project Setup & GitHub Management	Create Spring Boot project, configure dependencies, create package structure, set up MySQL, create `application.properties`, manage GitHub repository and merge pull requests	`setup-project`
Member 2	Patient Entity & Repository	Create `Patient` entity and `PatientRepository`	`patient-entity-repository`
Member 3	Patient DTO, Service & Controller	Create Patient DTOs, service interface/ implementation, and controller with CRUD APIs	`patient-service-controller`
Member 4	Doctor Entity & Repository	Create `Doctor` entity and `DoctorRepository`	`doctor-entity-repository`
Member 5	Doctor DTO, Service & Controller	Create Doctor DTOs, service interface/ implementation, and controller with CRUD APIs	`doctor-service-controller`
Member 6	Appointment Entity & Repository	Create `Appointment` entity, `AppointmentStatus` enum, and `AppointmentRepository`	`appointment-entity-repository`
Member 7	Appointment DTO, Service & Business Logic	Create appointment DTOs, booking logic, overlap validation, filter methods	`appointment-service`
Member 8	Appointment Controller & API Response	Create appointment controller, standard `ApiResponse` wrapper, and endpoint testing support	`appointment-controller-response`
Member 9	Validation, Exceptions, Postman & Documentation	Global exception handler, custom exceptions, validation testing, Postman collection, README and presentation slides	`exceptions-postman-docs`

🔀 GitHub Collaboration Workflow (9 Members)

👨‍💻 Member 1 – Initial Setup

1. Create Spring Boot project using Spring Initializr.
2. Add dependencies:
3. Spring Web
4. Spring Data JPA
5. MySQL Driver
6. Lombok
7. Validation
8. Create GitHub repository.
9. Push initial project to `main` branch.
10. Add all members as collaborators.

👥 All Members – Clone and Create Branch

```
git clone <repository-url>
cd hospital-appointment-system
git checkout -b your-branch-name
```

💾 Commit Changes

```
git add .
git commit -m "Completed patient entity and repository"
git push origin your-branch-name
```

🔁 Create Pull Request

- Open GitHub.
- Create Pull Request from your branch to `main`.
- Member 1 reviews and merges.

🔄 Update Local Code

```
git checkout main
git pull origin main
git checkout your-branch-name
git merge main
```

📝 Detailed Task Instructions for Each Member

Member 1 – Project Setup & Repository Manager

Tasks:

- Create project.
- Configure `application.properties`.
- Create packages.
- Resolve merge conflicts.
- Maintain clean project structure.

Deliverables:

- Working starter project.
- GitHub repository.
- Database configuration.

Member 2 – Patient Entity & Repository

Tasks:

- Create `Patient.java`.
- Add JPA annotations.
- Create `PatientRepository.java`.

Deliverables:

- Entity with validations and relationships.
- Repository interface.

Member 3 – Patient Module APIs

Tasks:

- Create request/response DTOs.
- Create service interface and implementation.
- Create `PatientController`.

Deliverables:

- Full Patient CRUD APIs.

Member 4 – Doctor Entity & Repository

Tasks:

- Create `Doctor.java`.
- Create `DoctorRepository.java`.

Deliverables:

- Doctor entity and repository.

Member 5 – Doctor Module APIs

Tasks:

- Create Doctor DTOs.
- Create service and implementation.
- Create `DoctorController`.

Deliverables:

- Full Doctor CRUD APIs.

Member 6 – Appointment Entity & Repository

Tasks:

- Create `Appointment.java`.
- Create `AppointmentStatus.java` enum.
- Create `AppointmentRepository.java`.

Deliverables:

- Appointment entity and repository methods.

Member 7 – Appointment Business Logic

Tasks:

- Create Appointment DTOs.
- Implement booking logic.
- Prevent overlapping appointments.
- Add filter methods.

Deliverables:

- Appointment service and business logic.

Member 8 – Appointment Controller & Standard Responses

Tasks:

- Create `AppointmentController`.
- Create `ApiResponse<T>` class.
- Ensure all APIs return consistent JSON.

Deliverables:

- Appointment endpoints.
- Standardized API responses.

Member 9 – Validation, Exceptions & Documentation

Tasks:

- Create custom exceptions.
- Create `GlobalExceptionHandler`.
- Build Postman collection.
- Write README.
- Prepare presentation.

Deliverables:

- Robust error handling.
- Postman collection.
- Documentation.

🧪 Integration Testing Strategy

After all Pull Requests are merged: 1. Run the application. 2. Test Patient APIs. 3. Test Doctor APIs. 4. Test Appointment booking. 5. Test overlapping rejection. 6. Test validation errors. 7. Export Postman collection.

🎤 Presentation Responsibility

Member	Explains
Member 1	Project architecture and GitHub collaboration
Member 2	Patient entity and relationships
Member 3	Patient APIs
Member 4	Doctor entity
Member 5	Doctor APIs
Member 6	Appointment entity
Member 7	Overlap prevention logic
Member 8	Appointment APIs and JSON responses
Member 9	Validation, exception handling, Postman demo

Member 1	Project setup, database configuration, common response classes
Member 2	Patient module (Entity, DTO, Repository, Service, Controller)
Member 3	Doctor module (Entity, DTO, Repository, Service, Controller)
Member 4	Appointment module and overlapping logic
Member 5	Exception handling, validation, Postman testing, documentation

---

🗂️ Project Structure

```
hospital-appointment-system/
├── src/main/java/com/example/hospital/
│   ├── HospitalApplication.java
│   ├── controller/
│   │   ├── PatientController.java
│   │   ├── DoctorController.java
│   │   └── AppointmentController.java
│   ├── service/
│   │   ├── PatientService.java
│   │   ├── DoctorService.java
│   │   ├── AppointmentService.java
│   │   └── impl/
│   │       ├── PatientServiceImpl.java
```

```
│   │       ├── DoctorServiceImpl.java
│   │       └── AppointmentServiceImpl.java
│   ├── repository/
│   │   ├── PatientRepository.java
│   │   ├── DoctorRepository.java
│   │   └── AppointmentRepository.java
│   ├── entity/
│   │   ├── Patient.java
│   │   ├── Doctor.java
│   │   ├── Appointment.java
│   │   └── AppointmentStatus.java
│   ├── dto/
│   │   ├── patient/
│   │   ├── doctor/
│   │   └── appointment/
│   ├── exception/
│   │   ├── ResourceNotFoundException.java
│   │   ├── AppointmentConflictException.java
│   │   └── GlobalExceptionHandler.java
│   └── payload/
│       └── ApiResponse.java
└── src/main/resources/
    └── application.properties
```

🛠️ Step-by-Step Implementation Guide

Step 1 – Create Spring Boot Project

Use:

- Spring Web
- Spring Data JPA
- MySQL Driver
- Lombok
- Validation

Step 2 – Configure Database

application.properties

```
spring.datasource.url=jdbc:mysql://localhost:3306/hospital_db
spring.datasource.username=root
spring.datasource.password=1234

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

server.port=8080
```

Step 3 – Create Entities

Patient

Stores patient details.

Doctor

Stores doctor details and specialization.

Appointment

Stores booking information with date, time, patient, and doctor.

Step 4 – Create DTOs

PatientRequestDTO

Used to create/update patient.

PatientResponseDTO

Returned to client.

DoctorRequestDTO

Used to create/update doctor.

DoctorResponseDTO

Returned to client.

AppointmentRequestDTO

Contains:

- patientId
- doctorId
- appointmentDate
- startTime
- endTime
- notes

AppointmentResponseDTO

Contains full appointment information.

Step 5 – Create Repositories

PatientRepository

```
Optional<Patient> findByEmail(String email);
```

DoctorRepository

```
Optional<Doctor> findByEmail(String email);
```

AppointmentRepository

```
List<Appointment> findByDoctorId(Long doctorId);
List<Appointment> findByAppointmentDate(LocalDate date);
List<Appointment> findByDoctorIdAndAppointmentDate(Long doctorId, LocalDate
date);
```

Step 6 – Create Service Interfaces

Methods:

- create

- getAll
- getById
- update
- delete

Appointment service also includes:

- getByDoctor
- getByDate
- getByDoctorAndDate

Step 7 – Implement Business Logic

Appointment Booking Logic

1. Validate patient exists.
2. Validate doctor exists.
3. Validate startTime < endTime.
4. Load existing appointments for doctor on the same date.
5. Check overlap.
6. Save appointment.

Overlap Condition

```
newStart < existingEnd AND newEnd > existingStart
```

Example:

- Existing: 10:00 – 11:00
- New: 10:30 – 11:30 ❌ Overlap
- New: 11:00 – 12:00 ✅ Allowed

Step 8 – Create Controllers

PatientController

Base URL: `/api/patients`

DoctorController

Base URL: `/api/doctors`

AppointmentController

Base URL: `/api/appointments`

Step 9 – Add Validation

Examples:

- `@NotBlank`
- `@Email`
- `@NotNull`
- `@FutureOrPresent`

Step 10 – Global Exception Handling

Handle:

- Resource not found
- Validation errors
- Appointment conflicts
- Generic exceptions

Step 11 – Standard API Response

```
{
  "timestamp": "2026-05-19T10:00:00",
  "status": 200,
  "message": "Patient created successfully",
  "data": {
    "id": 1,
    "firstName": "Nawodya"
  }
}
```

Step 12 – Postman Testing

Create collection with:

- Patient APIs

- Doctor APIs
- Appointment APIs
- Validation tests
- Conflict tests

📡 API Endpoint Summary

Patient APIs

Method	Endpoint	Description
POST	/api/patients	Create patient
GET	/api/patients	Get all patients
GET	/api/patients/{id}	Get patient by ID
PUT	/api/patients/{id}	Update patient
DELETE	/api/patients/{id}	Delete patient

Doctor APIs

Method	Endpoint	Description
POST	/api/doctors	Create doctor
GET	/api/doctors	Get all doctors
GET	/api/doctors/{id}	Get doctor by ID
PUT	/api/doctors/{id}	Update doctor
DELETE	/api/doctors/{id}	Delete doctor

Appointment APIs

Method	Endpoint	Description
POST	/api/appointments	Book appointment
GET	/api/appointments	Get all appointments
GET	/api/appointments/{id}	Get appointment by ID
PUT	/api/appointments/{id}	Update appointment
DELETE	/api/appointments/{id}	Delete appointment

Method	Endpoint	Description
GET	/api/appointments/doctor/{doctorId}	Filter by doctor
GET	/api/appointments/date/{date}	Filter by date
GET	/api/appointments/doctor/{doctorId}/date/{date}	Filter by doctor and date

---

🧪 Sample JSON Requests

Create Patient

```
{
  "firstName": "Nawodya",
  "lastName": "Rashmina",
  "email": "nawodya@example.com",
  "phoneNumber": "0771234567",
  "dateOfBirth": "2003-05-15",
  "gender": "Male"
}
```

Create Doctor

```
{
  "firstName": "Kamal",
  "lastName": "Perera",
  "specialization": "Cardiology",
  "email": "kamal@example.com",
  "phoneNumber": "0711234567"
}
```

Book Appointment

```
{
  "patientId": 1,
  "doctorId": 1,
  "appointmentDate": "2026-05-25",
  "startTime": "10:00:00",
  "endTime": "10:30:00",
  "notes": "Chest pain consultation"
}
```

🧠 Important Business Logic

Prevent Overlapping Appointments

The same doctor cannot have two appointments at the same time.

Invalid Example

- Appointment 1: 10:00–10:30
- Appointment 2: 10:15–10:45 ❌ Rejected

Valid Example

- Appointment 1: 10:00–10:30
- Appointment 2: 10:30–11:00 ✅ Allowed

🎁 Bonus Features

- Appointment status tracking
- Search doctors by specialization
- Pagination
- Sorting
- Swagger/OpenAPI documentation
- Soft delete

🎤 Final Demonstration Checklist

Functional Requirements

- [ ] Create Patient
- [ ] View All Patients
- [ ] Update Patient
- [ ] Delete Patient
- [ ] Create Doctor
- [ ] View All Doctors
- [ ] Update Doctor
- [ ] Delete Doctor
- [ ] Book Appointment
- [ ] Prevent Overlapping Appointments
- [ ] Filter by Date
- [ ] Filter by Doctor

Technical Requirements

- [ ] DTOs
- [ ] Validation
- [ ] Exception Handling
- [ ] Proper Layered Architecture
- [ ] Clean Code
- [ ] MySQL Integration

Presentation Requirements

- [ ] Postman Collection Ready
- [ ] Each Member Can Explain Their Part
- [ ] Code Runs Without Errors

🏆 Tips to Get Full Marks

1. Use DTOs everywhere.
2. Validate all request fields.
3. Return clean JSON responses.
4. Implement real business logic.
5. Use proper exception handling.
6. Keep package structure organized.
7. Test all APIs in Postman.
8. Ensure all members understand the system.

📅 Suggested 3-Day Plan

Day 1

- Create project
- Configure database
- Create entities and repositories

Day 2

- DTOs
- Services
- Controllers
- Validation

Day 3

- Exception handling
- Business logic
- Postman testing
- Final presentation

🎯 Evaluation Mapping

Criteria	How to Score High
CRUD Implementation	Complete all endpoints
Entity Relationships	Proper `@OneToMany` and `@ManyToOne`
API Design	RESTful naming
Validation & Error Handling	`@Valid` + global handler
Business Logic	Overlap prevention
Code Structure	Clean packages and DTOs
Demonstration	Working Postman collection

🚀 Recommended Next Steps

1. Generate the project using the Master Prompt.
2. Divide tasks among 5 members.
3. Merge code into Git.
4. Test all endpoints.
5. Practice the final demo.

This guide contains everything your team needs to build and present a professional Spring Boot Hospital Appointment System and achieve high marks.